TESLA 2010-2022- Stock Price Prediction using eXtreme Gradient Boosting (high accuracy on predictions)

Importing Libararies

In [1]:
```
!pip install xgboost
```

```
Requirement already satisfied: xgboost in c:\users\home\anaconda3\lib\site-packages
(3.2.0)
Requirement already satisfied: numpy in c:\users\home\anaconda3\lib\site-packages (f
rom xgboost) (1.26.4)
Requirement already satisfied: scipy in c:\users\home\anaconda3\lib\site-packages (f
rom xgboost) (1.13.1)
```

In [3]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sb

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from xgboost import XGBClassifier
from sklearn import metrics

import warnings
warnings.filterwarnings('ignore')
```

Importing DataSet

In [7]:
```
df=pd.read_csv('TSLA.csv')
df
```

Out[7]:

	Date	Open	High	Low	Close	Adj Close	Volume
0	6/29/2010	3.800000	5.000000	3.508000	4.778000	4.778000	93831500
1	6/30/2010	5.158000	6.084000	4.660000	4.766000	4.766000	85935500
2	7/1/2010	5.000000	5.184000	4.054000	4.392000	4.392000	41094000
3	7/2/2010	4.600000	4.620000	3.742000	3.840000	3.840000	25699000
4	7/6/2010	4.000000	4.000000	3.166000	3.222000	3.222000	34334500
...	...	...	...	...	...	...	...
2951	3/18/2022	874.489990	907.849976	867.390015	905.390015	905.390015	33408500
2952	3/21/2022	914.979980	942.849976	907.090027	921.159973	921.159973	27327200
2953	3/22/2022	930.000000	997.859985	921.750000	993.979980	993.979980	35289500
2954	3/23/2022	979.940002	1040.699951	976.400024	999.109985	999.109985	40225400
2955	3/24/2022	1009.729980	1024.489990	988.799988	1013.919983	1013.919983	22901900

2956 rows × 7 columns

In [13]:
```
df.shape
```

Out[13]: (2956, 7)

In [15]:
```
df.describe()
```

Out[15]:

	Open	High	Low	Close	Adj Close	Volume
count	2956.000000	2956.000000	2956.000000	2956.000000	2956.000000	2.956000e+03
mean	138.691296	141.771603	135.425953	138.762183	138.762183	3.131449e+07
std	250.044839	255.863239	243.774157	250.123115	250.123115	2.798383e+07
min	3.228000	3.326000	2.996000	3.160000	3.160000	5.925000e+05
25%	19.627000	20.402000	19.127500	19.615000	19.615000	1.310288e+07
50%	46.656999	47.487001	45.820002	46.545000	46.545000	2.488680e+07
75%	68.057001	69.357500	66.911501	68.103998	68.103998	3.973875e+07
max	1234.410034	1243.489990	1217.000000	1229.910034	1229.910034	3.046940e+08

In [17]:
```
df.info()
```

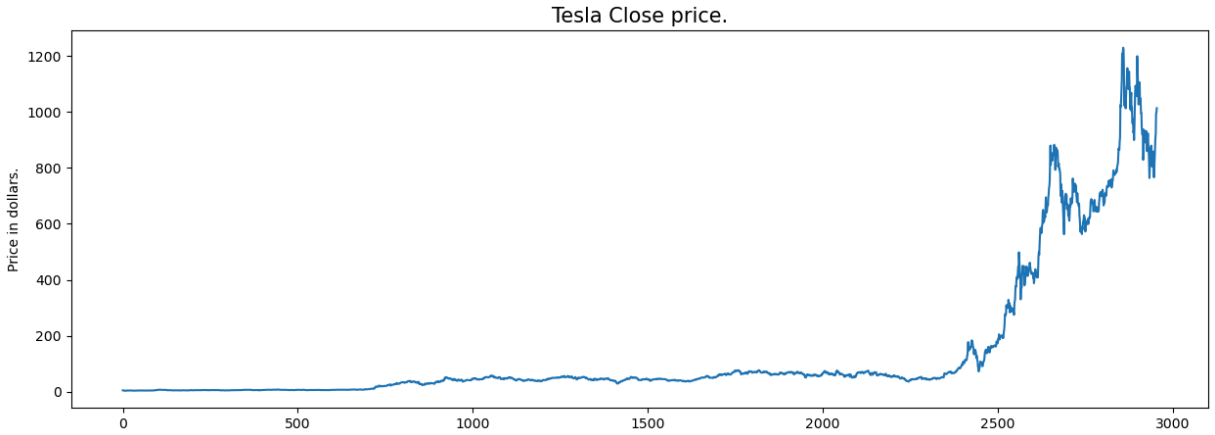
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2956 entries, 0 to 2955
Data columns (total 7 columns):
 #   Column     Non-Null Count  Dtype  
---  ------     --------------  -----  
 0   Date       2956 non-null   object 
 1   Open       2956 non-null   float64
 2   High       2956 non-null   float64
 3   Low        2956 non-null   float64
 4   Close      2956 non-null   float64
 5   Adj Close  2956 non-null   float64
 6   Volume     2956 non-null   int64  
dtypes: float64(5), int64(1), object(1)
memory usage: 161.8+ KB
```

In [ ]:

Exploratory Data Analysis

While performing the EDA of the Tesla Stock Price data we will analyze how prices of the stock have moved over the period of time and how the end of the quarters affects the prices of the stock.

In [21]:
```
plt.figure(figsize=(15,5))
plt.plot(df['Close'])
plt.title('Tesla Close price.', fontsize=15)
plt.ylabel('Price in dollars.')
plt.show()
```



The prices of tesla stocks are showing an upward trend as depicted by the plot of the closing price of the stocks.

In [ ]:

Checking if 'Close' and 'Adj Close' col are same and dropping the redundant col

In [25]:
```
df.head()
```

Out[25]:

	Date	Open	High	Low	Close	Adj Close	Volume
0	6/29/2010	3.800	5.000	3.508	4.778	4.778	93831500
1	6/30/2010	5.158	6.084	4.660	4.766	4.766	85935500
2	7/1/2010	5.000	5.184	4.054	4.392	4.392	41094000
3	7/2/2010	4.600	4.620	3.742	3.840	3.840	25699000
4	7/6/2010	4.000	4.000	3.166	3.222	3.222	34334500

In [27]:
```
df[df['Close'] == df['Adj Close']].shape ## checking
```

Out[27]:
```
(2956, 7)
```

In [29]:
```
df=df.drop(['Adj Close'],axis=1) ## dropping 'Adj Close'
```

In [31]:
```
df.head()
```

Out[31]:

	Date	Open	High	Low	Close	Volume
0	6/29/2010	3.800	5.000	3.508	4.778	93831500
1	6/30/2010	5.158	6.084	4.660	4.766	85935500
2	7/1/2010	5.000	5.184	4.054	4.392	41094000
3	7/2/2010	4.600	4.620	3.742	3.840	25699000
4	7/6/2010	4.000	4.000	3.166	3.222	34334500

Checking for null values

In [34]:
```
df.isnull().sum()
```

Out[34]:
```
Date      0
Open      0
High      0
Low       0
Close     0
Volume    0
dtype: int64
```

In [ ]:
```

```

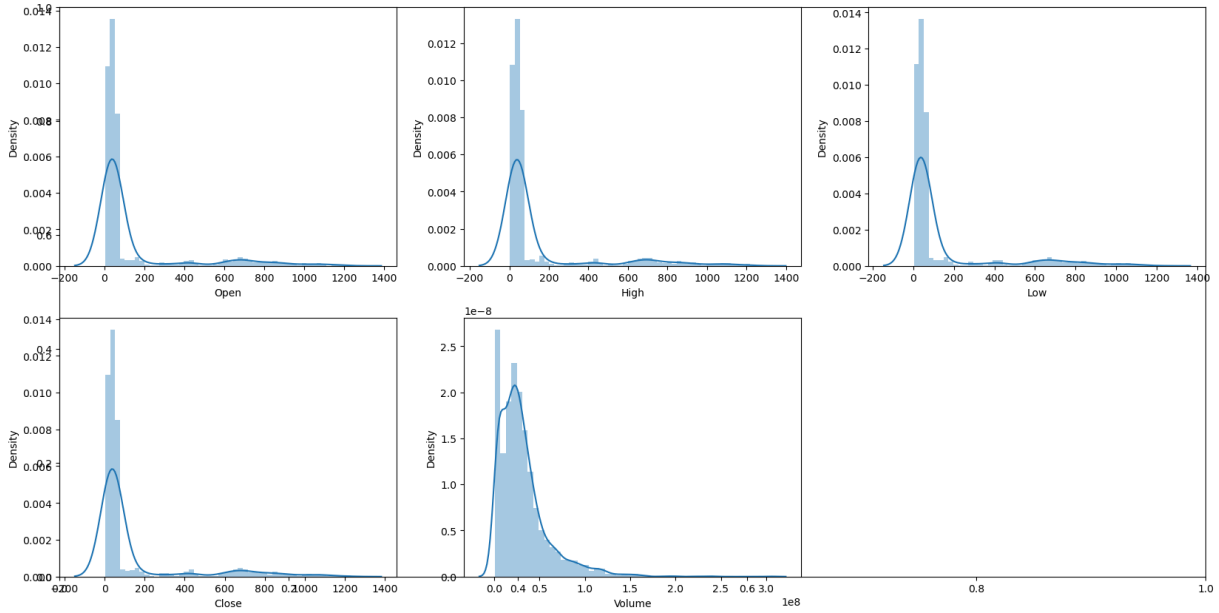
Distribution plot for the continuous features

In [42]:
```
features = ['Open', 'High', 'Low', 'Close', 'Volume']

plt.subplots(figsize=(20,10))

for i, col in enumerate(features):
  plt.subplot(2,3,i+1)
```

```
  sb.distplot(df[col])
plt.show()
```

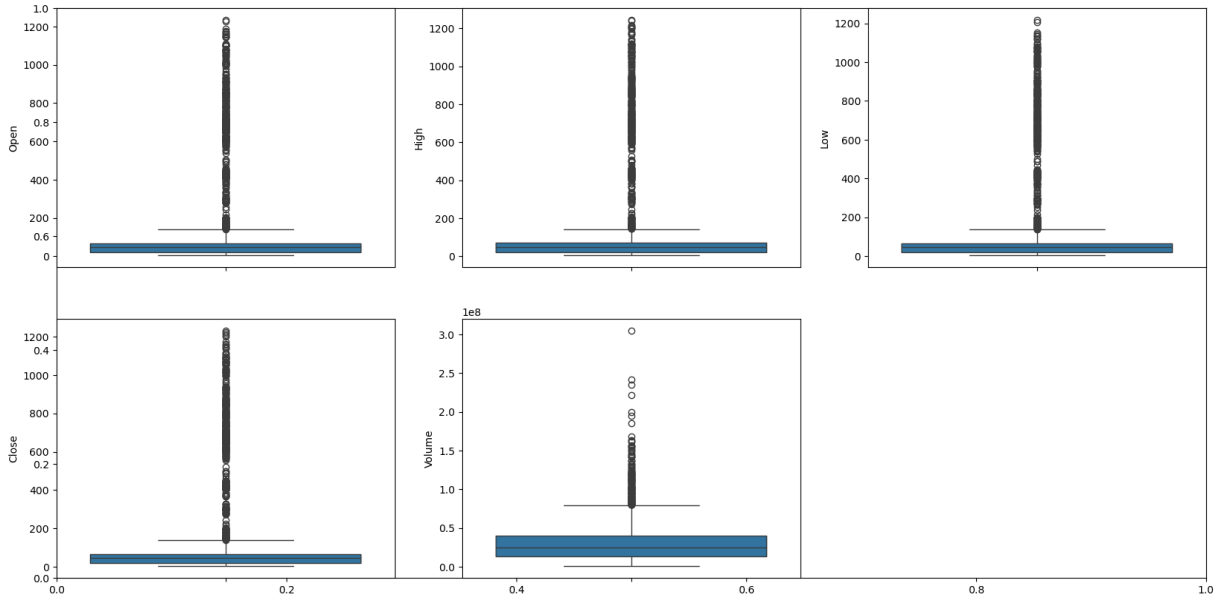


In [ ]:
```
In the distribution plot of OHLC data, we can see most values concentrate near lowe
Peak on left and long tail on right which is quite common in stock market data.
```

In [ ]:
```
```

In [ ]:
```
## Outliers
```

In [44]:
```
plt.subplots(figsize=(20,10))
for i, col in enumerate(features):
  plt.subplot(2,3,i+1)
  sb.boxplot(df[col])
plt.show()
```



Small Box Near Bottom - most stock prices are concentrated in lower ranges Long Upper Tail - some very high stock prices exist indicating Positive (Right) Skewness Many Outliers exsists - black circles above whiskers indicate: extreme stock price values

because of Tesla rapid growth periods, stock split,market rallies,volatility spikes

In [ ]:

Feature Engineering

Creating extra features to help in increase the performance of the model

In [49]:
```
splitted = df['Date'].str.split('/', expand=True)

df['month'] = splitted[0].astype('int')
df['day'] = splitted[1].astype('int')
df['year'] = splitted[2].astype('int')

df.head()
```

Out[49]:

	Date	Open	High	Low	Close	Volume	day	month	year
0	6/29/2010	3.800	5.000	3.508	4.778	93831500	29	6	2010
1	6/30/2010	5.158	6.084	4.660	4.766	85935500	30	6	2010
2	7/1/2010	5.000	5.184	4.054	4.392	41094000	1	7	2010
3	7/2/2010	4.600	4.620	3.742	3.840	25699000	2	7	2010
4	7/6/2010	4.000	4.000	3.166	3.222	34334500	6	7	2010

Added 3 more columns 'day', 'month' and 'year' from the 'Date' column which is initially provided in the data.

In [ ]:

In [51]:
```
df['is_quarter_end'] = np.where(df['month']%3==0,1,0)
df.head()
```

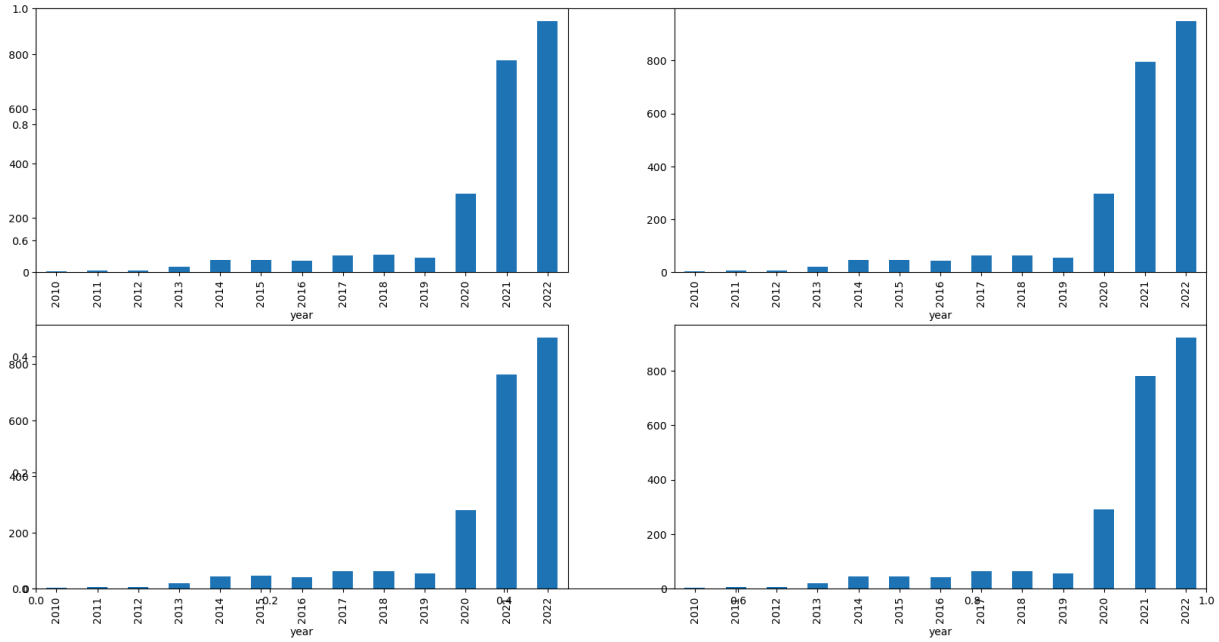
Out[51]:

	Date	Open	High	Low	Close	Volume	day	month	year	is_quarter_end
0	6/29/2010	3.800	5.000	3.508	4.778	93831500	29	6	2010	1
1	6/30/2010	5.158	6.084	4.660	4.766	85935500	30	6	2010	1
2	7/1/2010	5.000	5.184	4.054	4.392	41094000	1	7	2010	0
3	7/2/2010	4.600	4.620	3.742	3.840	25699000	2	7	2010	0
4	7/6/2010	4.000	4.000	3.166	3.222	34334500	6	7	2010	0

In [ ]:

In [53]:
```
data_grouped = df.drop('Date', axis=1).groupby('year').mean()
plt.subplots(figsize=(20,10))

for i, col in enumerate(['Open', 'High', 'Low', 'Close']):
  plt.subplot(2,2,i+1)
  data_grouped[col].plot.bar()
plt.show()
```



In [ ]:
```
Bar plots for yearly stock price trends. Tesla stock price started increasing since

Tesla stock experienced Strong Bullish Trend reflecting exponential growth, high in
```

In [ ]:
```

```

In [55]:
```
df.drop('Date', axis=1).groupby('is_quarter_end').mean()
```

Out[55]:

is_quarter_end	Open	High	Low	Close	Volume	day	
0	136.474690	139.523037	133.361644	136.531872	3.194378e+07	15.701987	6
1	143.073168	146.216652	139.506757	143.171146	3.007048e+07	15.736153	7

Tesla stock prices tend to be slightly higher during quarter-end periods.However trading activity remains high in both cases.

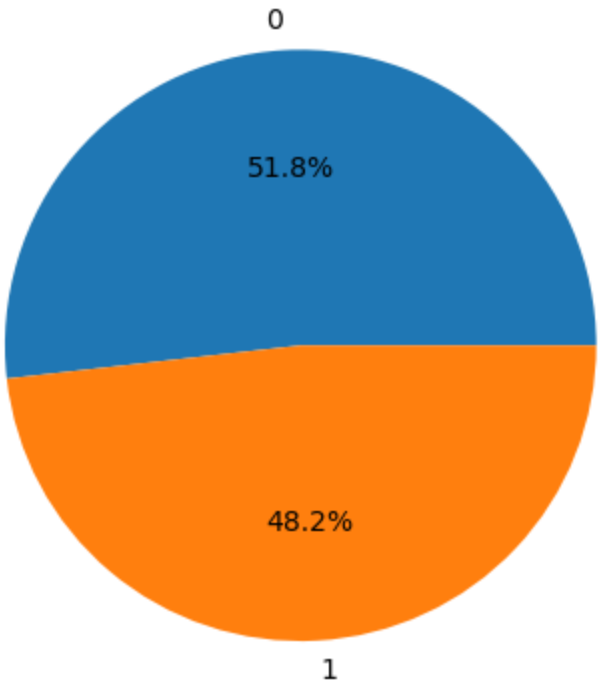
In [ ]:
```

```

Adding target feature to signal whether to buy or not - while balancing the same

In [58]:
```
df['open-close']  = df['Open'] - df['Close']
df['low-high']  = df['Low'] - df['High']
df['target'] = np.where(df['Close'].shift(-1) > df['Close'], 1, 0)
```

In [60]:
```
plt.pie(df['target'].value_counts().values,
        labels=[0, 1], autopct='%1.1f%%')
plt.show()
```



The dataset is almost Balanced as both classes are nearly equal which is good for ML models.

In [ ]:

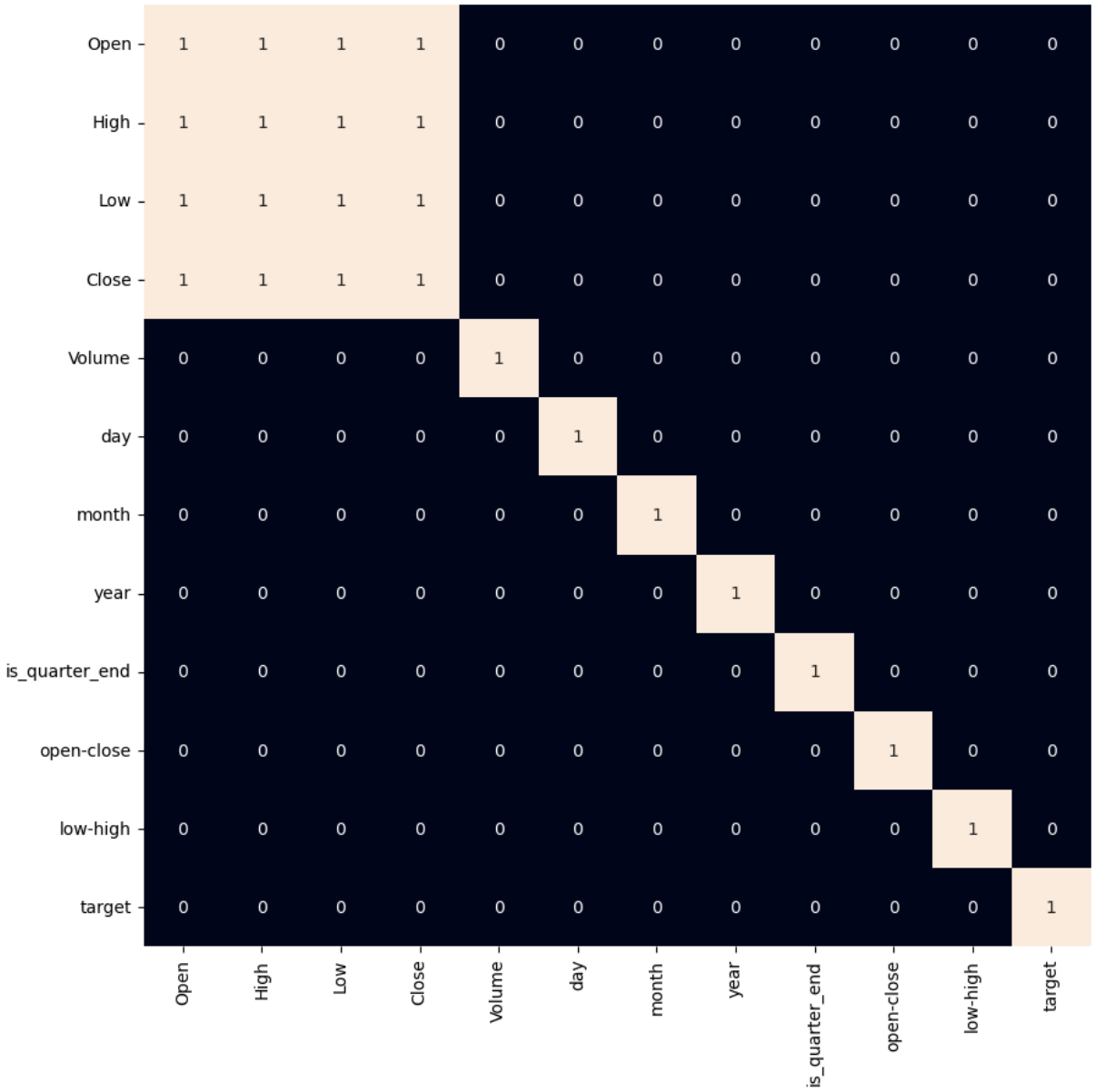
Heatmap to understand the correlation between the features

When we add features to our dataset we have to ensure that there are not highly correlated features as they do not help in the learning process of the algorithm resulting in curse of dimensionality

In [63]:
```
# As our concern is with the highly correlated features only, we will visualize  ou

plt.figure(figsize=(10, 10))

sb.heatmap(df.drop('Date', axis=1).corr() > 0.9, annot=True, cbar=False)
plt.show()
```



In [ ]:
```
From the above heatmap, we can say that there is a high correlation between OHLC.
The added features are not highly correlated with each other or previously provided
that we are good to go and build our model.
```

Data Splitting

In [68]:
```
features = df[['open-close', 'low-high', 'is_quarter_end']]
target = df['target']

X_train, X_valid, Y_train, Y_valid = train_test_split(features, target, test_size=0
print(X_train.shape, X_valid.shape)
```

```
(2660, 3) (296, 3)
```

In [ ]:
```

```

Normalisation

In [72]:
```
scaler = StandardScaler()
features = scaler.fit_transform(features)
```

In [ ]:
```

```

Model Development and evaluation metrics

In [75]:
```
models = [LogisticRegression(), SVC(kernel='poly', probability=True), XGBClassifier

for i in range(3):
  models[i].fit(X_train, Y_train)

  print(f'{models[i]} : ')
  print('Training Accuracy : ', metrics.roc_auc_score(Y_train, models[i].predict_pr
  print('Validation Accuracy : ', metrics.roc_auc_score(Y_valid, models[i].predict_
  print()
```

```
LogisticRegression() :
Training Accuracy :  0.5144645206933145
Validation Accuracy :  0.5385989010989012

SVC(kernel='poly', probability=True) :
Training Accuracy :  0.48536568800848956
Validation Accuracy :  0.5230998168498169

XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric=None, feature_types=None,
              feature_weights=None, gamma=None, grow_policy=None,
              importance_type=None, interaction_constraints=None,
              learning_rate=None, max_bin=None, max_cat_threshold=None,
              max_cat_to_onehot=None, max_delta_step=None, max_depth=None,
              max_leaves=None, min_child_weight=None, missing=nan,
              monotone_constraints=None, multi_strategy=None, n_estimators=None,
              n_jobs=None, num_parallel_tree=None, ...) :
Training Accuracy :  0.9240990449239477
Validation Accuracy :  0.47685439560439563
```

In [ ]:
```
Among the three models, we have trained XGBClassifier has the highest performance b
between the training and the validation accuracy is too high. But in the case of th
```

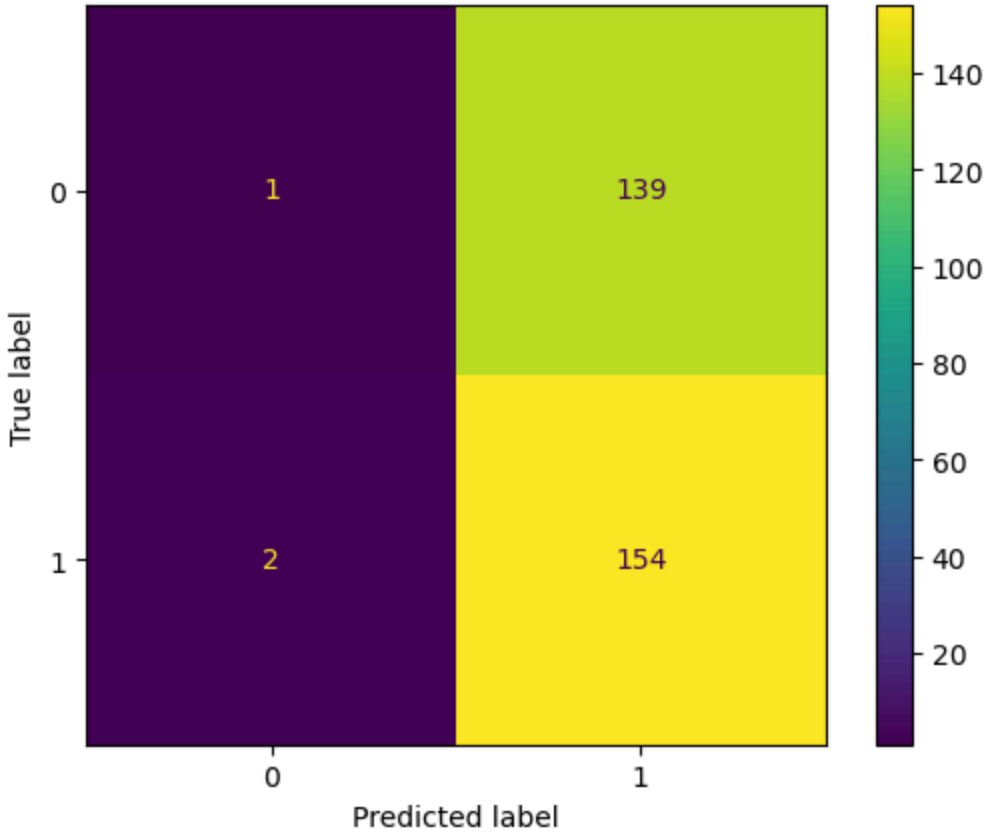
In [ ]:
```

```

Confusion Metrics

In [79]:
```
from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_estimator(models[0], X_valid, Y_valid)
plt.show()
```



The model correctly predicted the price would not go up only 1 time(True Negative - Top_left) The model predicted that the price would go up but actually it didnt. This is the biggest Error category (False Positive - Top_right,139) The model predicted the price wouldnt go p but it actually did (False - Negative,2) The model correctly predicted the price would go up 154 times (True Postives,Bottom_right,15)

In [ ]:

Conclusion: - The model is biased towards predicting 'price up' ('1'). - Overall accuracy (~50%)is misleading as the model is just 'guessing' the positive class.Possible reasons for this may be the lack of data or using a very simple model to perform such a complex task as Stock Market prediction. - For stock trading this model is risky,as its precision is low i.e when it says 'buy'its wrong nearly half times (139 false alarms compared to 154 correct ones).

In [ ]:

In [ ]: